

计算机组成原理第五次实验报告

实验名称：单周期 CPU 实验

学号：2410668

姓名：吕昊远

班次：张金、董前琨

一、实验目的

1. 理解 MIPS 指令结构与指令集中常用指令的功能和编码，并对这些指令进行归纳分类。
2. 熟悉 MIPS 体系的处理器结构，了解相关的各种概念。
3. 熟悉并掌握单周期 CPU 的原理和设计。
4. 进一步加强运用 verilog 语言进行电路设计的能力，为后续设计多周期 cpu 的实验打下基础。

二、实验内容

1. 阅读实验箱相关文档，学习单周期 CPU 与 MIPS 指令的基础功能与结构，将给出的代码配置为项目，生成位流文件烧录至实验箱进行基础功能的复现与验证。
2. 深入学习硬件中 CPU 的结构与工作原理，了解单周期 CPU 的运行逻辑。
3. 了解指令集与汇编的相关知识，设计新增的指令来解决特定问题，并绘制逻辑结构图。
4. 运用给出的指令与新添加的指令，实现斐波那契数列的计算。
5. 根据设计的改进方案，使用 verilog 修改核心模块与显示模块以及引脚约束文件，并编写仿真文件进行仿真波形验证，完成后按照步骤生成位流文件，将其导入实验箱烧录，在实验箱上对比验证各项功能。

三、实验原理图

依照指导书的原理图，我们可以知道单周期 CPU 的具体工作流程。CPU 在一个时钟周期内，根据 PC 值从指令 ROM 中读出相应的指令，将指令译码后再从寄存器堆中读出需要的操作数，送往 ALU 模块进行相应的运算。ALU 模块运算得到结果后，CPU 再根据指令类型分别进行相应的后续操作。

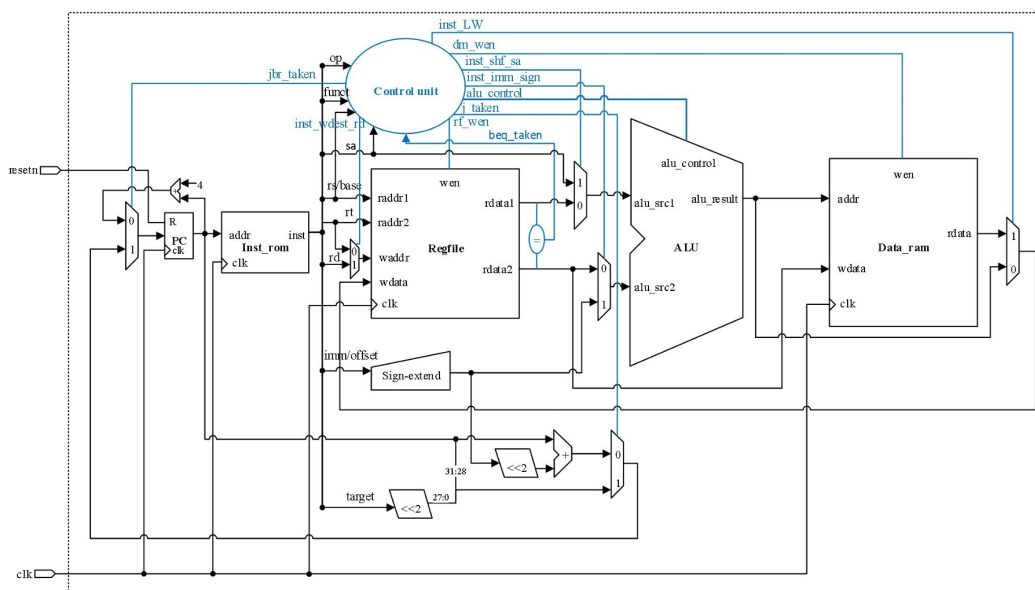


图 1: 单周期 CPU 模块示意图

如果是 store 指令，则 ALU 运算结果为数据存储的地址，就向数据 RAM 发出写请求，在下一个时钟上升沿真正写入到数据存储；如果是 load 指令，则 ALU 运算结果为数据存储的地址，根据该值从数据 RAM 中读出数据，送往寄存器堆根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中；如果不是 load/store 操作，若有写寄存器堆的操作，则直接将 ALU 运算结果送往寄存器堆，根据目的寄存器发出写请求，在下一个时钟上升沿真正写入到寄存器堆中；如果是分支跳转指令，则需要将结果写入到 pc 寄存器中。

在本实验中，我们需要使用单周期 CPU 完成斐波那契数列的计算，这里将给出的汇编示意图细化成具体实现时的每条汇编指令。主要分为 init 段的初始化和 loop 段的循环。原理图如下：

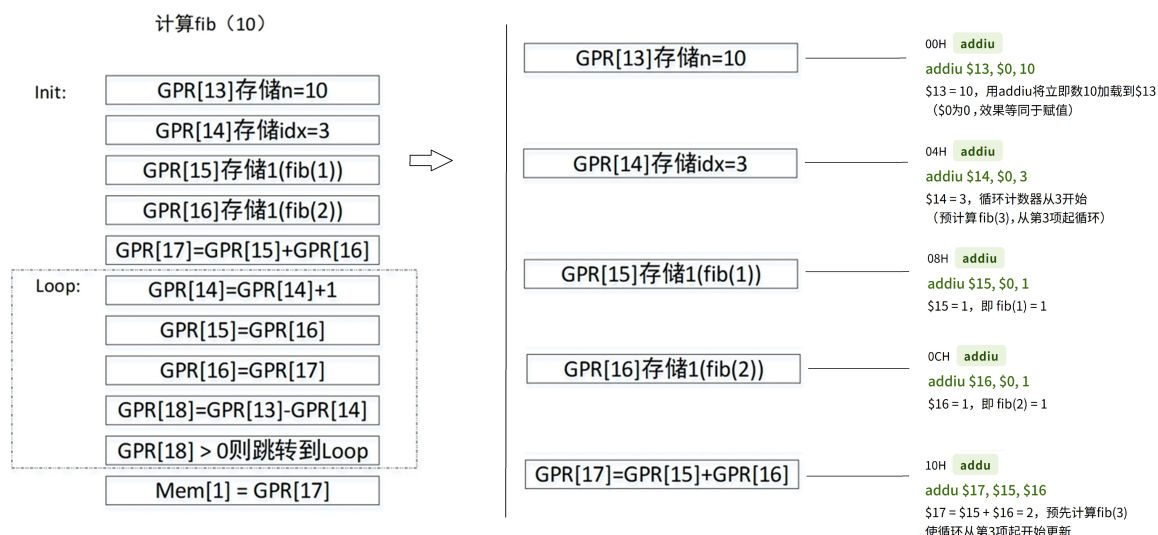


图 2: 初始化部分

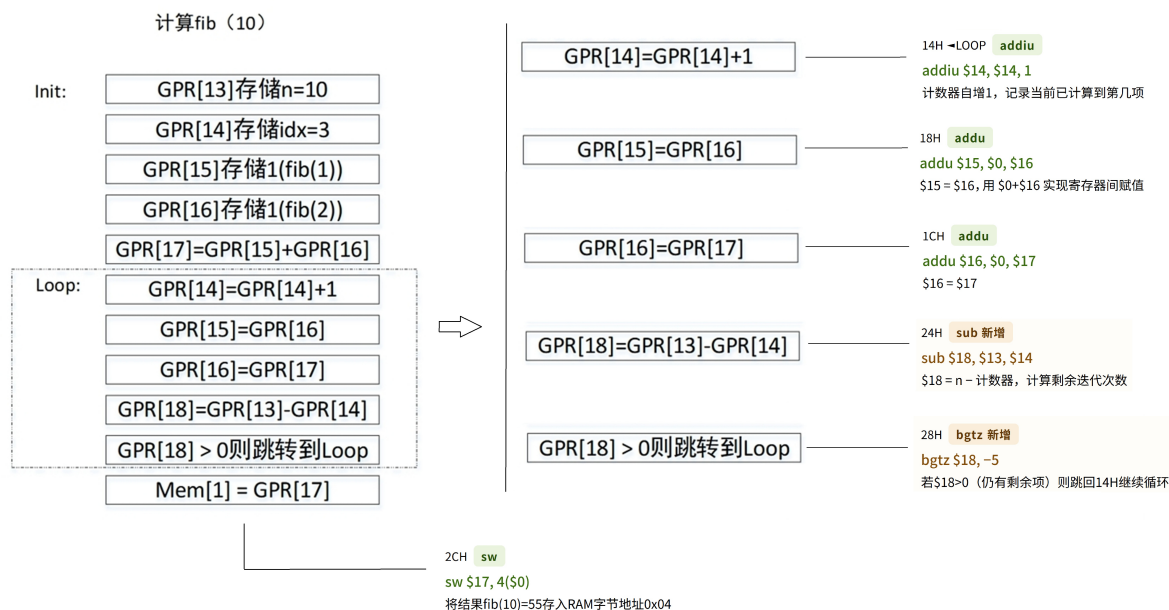


图 3: 循环体与结束

计算斐波那契数列的关键在于每一项等于前两项之和。采用滚动更新的策略，只使用三个寄存器 \$15、\$16、\$17 分别维护连续三项斐波那契的数值，每次循环将三个寄存器整体向前滚动一步，同时计算出新的下一项，如此反复直到计算到第 n 项为止。这种方式无需数组，仅用固定数量的寄存器即可完成计算。

初始化阶段将目标项数 10 存入 \$13 作为终止基准，计数器 \$14 初始化为 3，\$15、\$16 分别赋值为 fib(1) 和 fib(2)，并预先计算 fib(3) 存入 \$17，使三个寄存器在进入循环前就有一组符合我们要求的连续的三项数值。接着进入循环体，每次迭代先将计数器加 1，再依次将 \$16 移入 \$15、\$17 移入 \$16，最后将二者相加得到新的下一项存入 \$17，完成一步的推进。之后使用新增的 sub 指令计算目标项数与计数器的差值，再由新增的 bgtz 指令判断是否跳回循环入口继续执行，差值归零时则退出循环。退出循环时 \$17 中即为 fib(10) = 55（即 0x37）。通过 sw 指令将结果写入数据 RAM，随后跳转回起始地址，以便在板上或仿真中持续观察验证结果。

四、实验过程

1. 单周期 CPU 复现

使用 Vivado 创建项目文件，将老师提供的项目代码导入（为防止项目配置问题，此处使用文本级的复制粘贴进行导入）。

在仿真文件这部分，为了清晰看到内存数据，我们对 tb.v 进行了一定的修改，使其可以在运行完一遍指令后扫描各个内存来得到内存数据：

```

1 // .....
2
3 integer i;
4 initial begin
5     clk = 0;
6     resetn = 0;
7     rf_addr = 0;

```

```

8      mem_addr = 0;
9
10     #100;
11     resetn = 1;
12     #300;
13     // 扫描 mem 的数据
14     for (i = 0; i < 32; i = i + 1) begin // 循环遍历 32 个内存字地址
15         mem_addr = i * 4;
16         #10;
17         // 打印当前地址和读出的数据
18         $display("Time: %0t | MADDR: %h | MDATA: %h", $time, mem_addr, mem_data);
19     end
20     $finish;
21 end
22 always #5 clk=~clk;

```

经过仿真，我们得到以下波形：

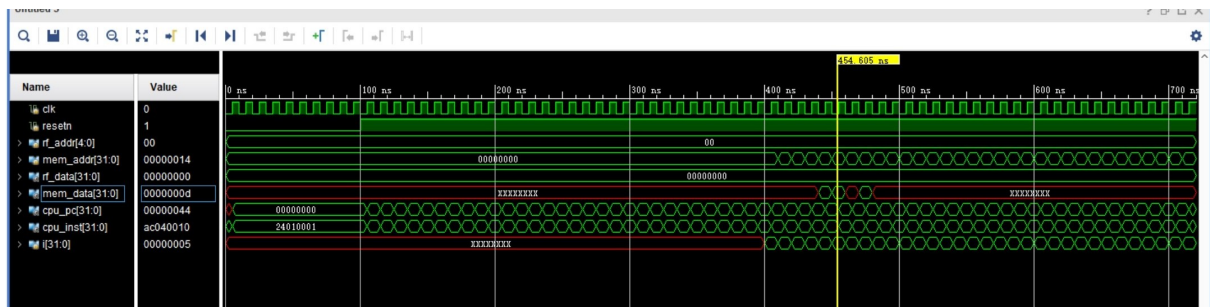


图 4: 仿真波形图

为了方便地展示数据，我们将内存地址与读出的数据打印在控制台上：

```

1  .....
2  Time: 440000 | MADDR: 0000000c | MDATA: xxxxxxxx
3  Time: 450000 | MADDR: 00000010 | MDATA: 00000004
4  Time: 460000 | MADDR: 00000014 | MDATA: 0000000d
5  Time: 470000 | MADDR: 00000018 | MDATA: xxxxxxxx
6  Time: 480000 | MADDR: 0000001c | MDATA: 00000000
7  Time: 490000 | MADDR: 00000020 | MDATA: xxxxxxxx
8  Time: 500000 | MADDR: 00000024 | MDATA: xxxxxxxx
9  .....

```

接下来继续添加约束文件。依次运行 Synthesis 与 Implementation，最后生成位流文件 Bitstream；代码项目生成完毕后，连接到实验箱，将位流文件烧录至开发板上，依照设计进行功能验证。

通过点按实验箱对应的按键让 CPU 单步执行，再使用 LCD 屏触控输入，得到以下测试结果：

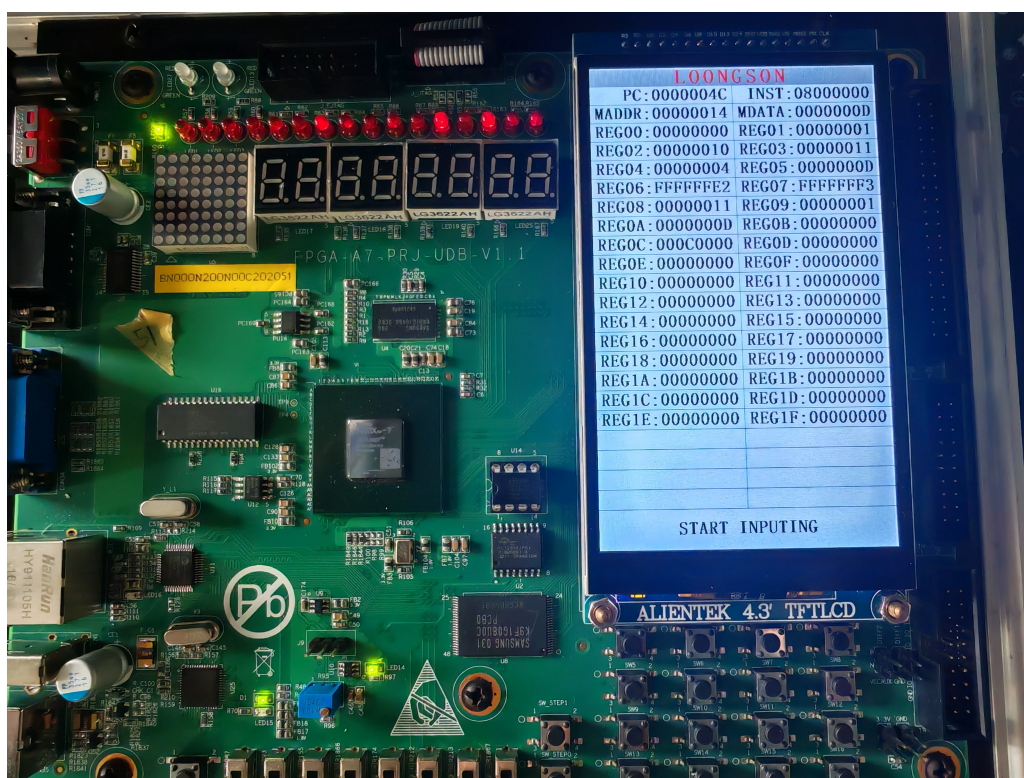


图 5: 实验箱运行结果

经过一次完整的循环后，查看内存与寄存器的值。经过验证，结果符合指导书中的指令流程，且与仿真波形的数据一致，结果均准确无误。

接下来将以此为基础，添加相应的指令并编写斐波那契数列的计算逻辑。

2. 斐波那契数列的计算

完成了单周期 CPU 的复现后，依据实验要求，新增指令并实现斐波那契数列的计算。在这部分，只需修改核心的单周期 CPU 模块 `single_cycle_cpu.v` 新增 `sub` 与 `bgtz` 这两项指令，并在 `inst_rom.v` 中编写计算 `fib(10)` 的汇编程序即可，其余部分无需修改。接下来，我们将详细讲解具体的实现流程。

i. 设计文件

首先在核心模块 `single_cycle_cpu.v` 中新增 `sub` 与 `bgtz` 新增这两项指令的实现：

```

1 // .....
2 // 新增指令译码
3 wire inst_SUB;
4 wire inst_BGTZ;
5 assign inst_SUB = op_zero & sa_zero & (funct == 6'b100010); // 有符号减法
6 assign inst_BGTZ = (op == 6'b000111); // 大于零跳转
7 // .....
8
9 // bgtz跳转条件：rs为有符号正数（最高位为0且值不为0）

```

```

10  wire bgtz_taken;
11  assign bgtz_taken = (~rs_value[31]) & (!rs_value); // rs > 0 (有符号)
12
13  // 综合所有跳转信号
14  assign jbr_taken = j_taken
15                      | (inst_BEQ & beq_taken)
16                      | (inst_BNE & bne_taken)
17                      | (inst_BGTZ & bgtz_taken); // 新增 bgtz 跳转
18  assign jbr_target = j_taken ? j_target : br_target;
19  // .....
20
21  // ALU 控制信号生成 (inst_sub 新增 inst_SUB)
22  // .....
23  assign inst_sub = inst_SUBU | inst_SUB; // SUB 与 SUBU 共用减法 ALU
24  // .....
25
26  // inst_wdest_rd 新增 inst_SUB (SUB 结果写回 rd 寄存器)
27  assign inst_wdest_rd = inst_ADDU | inst_SUBU | inst_SUB | inst_SLT | inst_AND
28                      | inst_NOR | inst_OR | inst_XOR | inst_SLL | inst_SRL;

```

• sub 指令

sub 是 R 型有符号减法指令，格式与 addu、subu 等 R 型指令完全一致，区别仅在于功能码 funct 字段为 100010。它的运算逻辑是将 rs 与 rt 两个寄存器的值相减，然后将结果写回 rd。

在 CPU 中实现 sub 的较为简单：在译码阶段增加一条识别语句，将 funct == 100010 的 R 型指令识别为 inst_SUB；ALU 控制信号中将其并入已有的减法通路，即 inst_sub = inst_SUBU | inst_SUB；写回阶段将 inst_SUB 加入写 rd 寄存器的条件中。这部分修改都是在复用已有的减法逻辑，无需改动 ALU 本身。

• bgtz 指令

bgtz 是 I 型跳转指令，操作码为 000111，格式中 rt 字段固定为零，rs 字段指定被比较的寄存器，offset 字段为 16 位有符号偏移量。其语义是，若 rs 寄存器的值为有符号正数，则跳转到 $PC + \text{sign_ext}(\text{offset}) \times 4$ 。

判断“有符号正数”的条件是最高位为 0（非负）且值不为全零，对应 Verilog 表达式 $(rs[31]) \& (!rs)$ 。跳转目标地址的计算与 beq、bne 完全一致，因此这里直接复用了已有的分支目标地址逻辑 br_target，仅需将 bgtz_taken 信号并入综合跳转信号 jbr_taken 即可。

经过以上修改，模块已经实现了新增的两个指令，接下来则需要继续修改 inst_rom.v，编写斐波那契数列 fib(10) 的计算程序。

```

1  `timescale 1ns / 1ps
2
3  module inst_rom(
4      input      [4 :0] addr,
5      output reg [31:0] inst
6  );
7

```

```

8      wire [31:0] inst_rom[19:0];
9
10     // 初始化段 (00H ~ 10H) -----
11     assign inst_rom[ 0] = 32'h240D000A; // 00H: addiu $13,$0,10    | $13 = 10
12     assign inst_rom[ 1] = 32'h240E0003; // 04H: addiu $14,$0,3    | $14 = 3 (初始计
        数器)
13     assign inst_rom[ 2] = 32'h240F0001; // 08H: addiu $15,$0,1    | $15 = fib(1) =
        1
14     assign inst_rom[ 3] = 32'h24100001; // 0CH: addiu $16,$0,1    | $16 = fib(2) =
        1
15     assign inst_rom[ 4] = 32'h01F08821; // 10H: addu  $17,$15,$16 | $17 = fib(3) =
        2
16
17     // 循环段 (14H ~ 28H)
18     assign inst_rom[ 5] = 32'h25CE0001; // 14H: addiu $14,$14,1    | $14++
19     assign inst_rom[ 6] = 32'h00107821; // 18H: addu  $15,$0,$16 | $15 = $16
20     assign inst_rom[ 7] = 32'h00118021; // 1CH: addu  $16,$0,$17 | $16 = $17
21     assign inst_rom[ 8] = 32'h01F08821; // 20H: addu  $17,$15,$16 | $17 = $15+$16
22     assign inst_rom[ 9] = 32'h01AE9022; // 24H: sub   $18,$13,$14 | $18 = n - 计数
        器
23     assign inst_rom[10] = 32'h1E40FFFB; // 28H: bgtz  $18,-5      | 若$18>0则跳回
        14H
24     // 目标地址14H=0x14, bgtz处28H=0x28, offset = (0x14-0x28)/4 = -5
25
26     // 结束段 (2CH ~ 30H)
27     assign inst_rom[11] = 32'hAC110004; // 2CH: sw     $17,4($0)    | Mem[0x04] = fib
        (10) = 55
28     assign inst_rom[12] = 32'h08000000; // 30H: j      00H          | 跳回起始
29
30     // 未使用的指令槽填0
31     assign inst_rom[13] = 32'd0;
32     assign inst_rom[14] = 32'd0;
33     assign inst_rom[15] = 32'd0;
34     assign inst_rom[16] = 32'd0;
35     assign inst_rom[17] = 32'd0;
36     assign inst_rom[18] = 32'd0;
37     assign inst_rom[19] = 32'd0;
38
39     always @(*)
40     begin
41         case (addr)
42             5'd0 : inst <= inst_rom[0 ];
43             5'd1 : inst <= inst_rom[1 ];
44             5'd2 : inst <= inst_rom[2 ];
45             5'd3 : inst <= inst_rom[3 ];
46             5'd4 : inst <= inst_rom[4 ];
47             5'd5 : inst <= inst_rom[5 ];
48             5'd6 : inst <= inst_rom[6 ];
49             5'd7 : inst <= inst_rom[7 ];
50             5'd8 : inst <= inst_rom[8 ];

```

```

51         5'd9 : inst <= inst_rom[9 ];
52         5'd10: inst <= inst_rom[10];
53         5'd11: inst <= inst_rom[11];
54         5'd12: inst <= inst_rom[12];
55         default: inst <= 32'd0;
56     endcase
57 end
58 endmodule

```

在这部分，由于 MIPS 没有专用的赋值指令，所以寄存器间的数值传递都需要借助 `addu $rd, $0, $rs` 来实现。`$0` 恒为零，加上源寄存器的值就等效于把源寄存器的内容复制到目标寄存器。

具体实现上，在每次循环体里，先执行 `addu $15, $0, $16` 把旧的 `$16` 存入 `$15`，再执行 `addu $16, $0, $17` 把旧的 `$17` 存入 `$16`，最后 `addu $17, $15, $16` 将新的前两项相加得到下一项。这里我们需要注意运算的顺序，必须要先保存 `$16` 然后再覆盖它，否则原值就丢失了；同样地，`$17` 也要在被覆盖前先转移走。

循环结束的判断则依赖新增的两条指令：`sub $18, $13, $14` 计算目标项数与当前计数器的差，`bgtz $18, -5` 则用来检查差值是否仍大于零。差值大于零说明还未到达目标项，跳回循环继续；差值等于零时条件不成立，顺序执行到 `sw` 指令将 `$17` 中的结果写入内存，从而得到最终结果。

ii. 仿真文件

```

1  `timescale 1ns / 1ps
2
3  module tb;
4
5      reg clk;
6      reg resetn;
7      reg [4:0] rf_addr;
8      reg [31:0] mem_addr;
9
10     wire [31:0] rf_data;
11     wire [31:0] mem_data;
12     wire [31:0] cpu_pc;
13     wire [31:0] cpu_inst;
14
15     single_cycle_cpu uut (
16         .clk(clk),
17         .resetn(resetn),
18         .rf_addr(rf_addr),
19         .mem_addr(mem_addr),
20         .rf_data(rf_data),
21         .mem_data(mem_data),
22         .cpu_pc(cpu_pc),
23         .cpu_inst(cpu_inst)
24     );
25
26     initial begin
27         clk = 0;
28         resetn = 0;

```

```

29     rf_addr = 0;
30     mem_addr = 0;
31
32     #100;
33     resetn = 1;
34
35     #50;
36     rf_addr = 5'h11;
37     mem_addr = 32'h4;
38     #850;
39     $finish;
40 end
41 always #5 clk=~clk;
42 endmodule

```

我们在程序中把计算的结果存入了 RAM 字节地址 0x04 中，因此我们需要在波形中查看对应的寄存器与内存的值是否和我们的预期一致。

iii. 结果验证

运行仿真，我们得到以下波形：

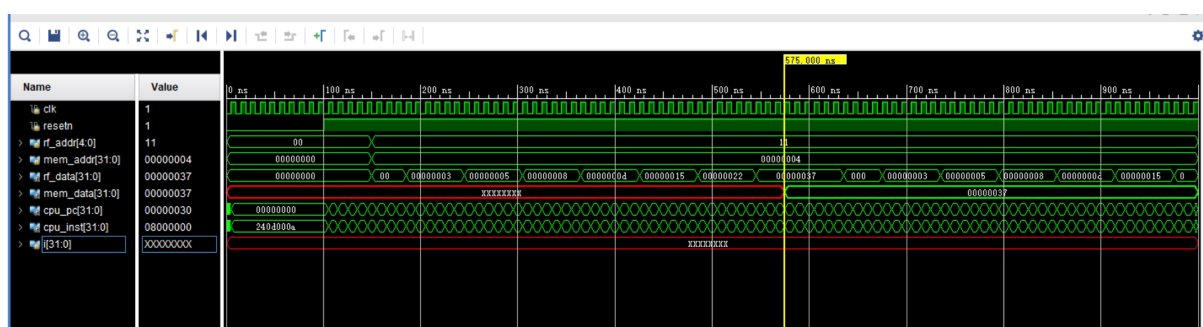


图 6: 仿真波形图

从仿真波形图中可以看出，内存 0x04 和寄存器 \$17 均为 0x37，即 $\text{fib}(10) = 0x37 = 55$ ，计算结果正确。接下来，继续生成位流文件并烧录至实验箱，在实验箱上进行功能验证。

点按实验箱上对应的按钮，进行单步执行。程序运行完成后与在 LCD 屏上查看各个寄存器的值，并通过输入地址来查看对应的内存的值，得到如下结果：

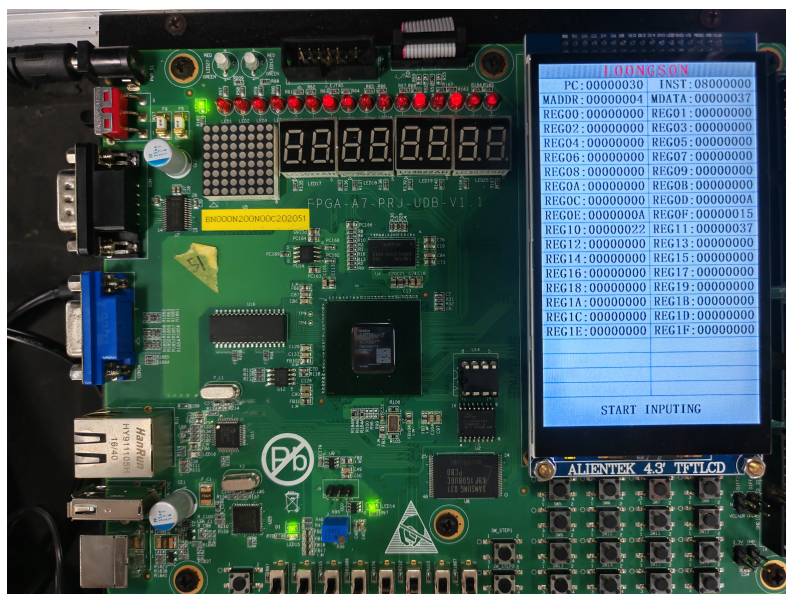


图 7: 实验箱运行结果

实验箱上的寄存器与内存的数值均准确无误，完美符合实验预期，验证了代码逻辑的正确性。

五、总结感想

本次实验中，首先通过提供的源码复现了基础的单周期 CPU。通过仿真波形与实验箱验证，学习了解了对应的原理与实现流程，在此基础上新增了两项指令，并在指令存储器中编写了斐波那契数列的计算逻辑，依照要求完成了本次实验。

通过这次实验，我了解了单周期 CPU 的工作原理和代码实现逻辑，对于 CPU 的基础结构有了进一步的理解。这一元件是程序执行的核心，也是现代计算机中最本质的基础硬件。在本次实验过程中，运用指令来实现我们所需的计算是最为重要的一部分，通过实践将问题拆解为一步步指令进行执行，这让我对于计算机的本质有了更加深入的认知；同时，在对仿真波形上进行相关的验证时，调试的一系列过程也锻炼了我的关联思维，对于如何全方位地进行测试与验证积累了宝贵的经验。